

EXPERIMENT 13

IMPLEMENTATION OF DIFFIE–HELLMAN KEY EXCHANGE

AIM:

To implement the Diffie–Hellman Key Exchange algorithm in C and establish a common secret key between two communicating parties over an insecure channel.

ALGORITHM:

1. Choose a prime number P and a primitive root G.
2. Alice selects a private key a.
3. Bob selects a private key b.
4. Alice calculates her public key:

$$A = G^a \text{ mod } P$$

5. Bob calculates his public key:

$$B = G^b \text{ mod } P$$

6. Alice calculates the shared secret key:

$$K1 = B^a \text{ mod } P$$

7. Bob calculates the shared secret key:

$$K2 = A^b \text{ mod } P$$

8. Since:

$$K1 = K2$$

both parties obtain the same secret key.

PROGRAM:

```
#include <stdio.h>
```

```
/* Function to calculate (base^exp) mod mod */
```

```
long long powerMod(long long base,
```

```
        long long exp,
        long long mod)
{
    long long result = 1;

    base = base % mod;

    while (exp > 0)
    {
        if (exp % 2 == 1)
        {
            result = (result * base) % mod;
        }

        base = (base * base) % mod;
        exp = exp / 2;
    }

    return result;
}

int main()
{
```

```
long long P, G;

long long privateA, privateB;

long long publicA, publicB;

long long sharedA, sharedB;


printf("DIFFIE-HELLMAN KEY EXCHANGE\n");

printf("-----\n");


printf("Enter prime number P: ");

scanf("%lld", &P);


printf("Enter primitive root G: ");

scanf("%lld", &G);


printf("Enter Alice's private key: ");

scanf("%lld", &privateA);


printf("Enter Bob's private key: ");

scanf("%lld", &privateB);


/* Alice calculates public key */

publicA = powerMod(G, privateA, P);
```

```
/* Bob calculates public key */

publicB = powerMod(G, privateB, P);


/* Alice calculates shared secret */

sharedA = powerMod(publicB, privateA, P);


/* Bob calculates shared secret */

sharedB = powerMod(publicA, privateB, P);


printf("\nPublic Keys\n");

printf("Alice's Public Key = %lld\n", publicA);

printf("Bob's Public Key  = %lld\n", publicB);


printf("\nShared Secret Keys\n");

printf("Alice's Shared Key = %lld\n", sharedA);

printf("Bob's Shared Key  = %lld\n", sharedB);


if (sharedA == sharedB)

    printf("\nKey Exchange Successful!\n");

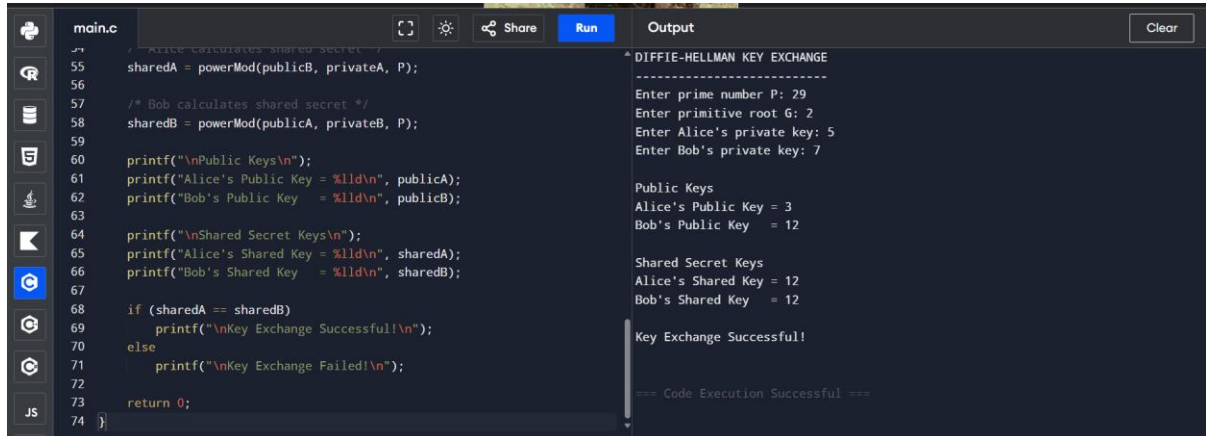
else

    printf("\nKey Exchange Failed!\n");


return 0;
```

}

OUTPUT:



```
main.c
55 sharedA = powerMod(publicB, privateA, P);
56
57 /* Bob calculates shared secret */
58 sharedB = powerMod(publicA, privateB, P);
59
60 printf("\nPublic Keys\n");
61 printf("Alice's Public Key = %ld\n", publicA);
62 printf("Bob's Public Key   = %ld\n", publicB);
63
64 printf("\nShared Secret Keys\n");
65 printf("Alice's Shared Key = %ld\n", sharedA);
66 printf("Bob's Shared Key   = %ld\n", sharedB);
67
68 if (sharedA == sharedB)
69     printf("\nKey Exchange Successful!\n");
70 else
71     printf("\nKey Exchange Failed!\n");
72
73 return 0;
74 }
```

DIFFIE-HELLMAN KEY EXCHANGE

Enter prime number P: 29

Enter primitive root G: 2

Enter Alice's private key: 5

Enter Bob's private key: 7

Public Keys

Alice's Public Key = 3

Bob's Public Key = 12

Shared Secret Keys

Alice's Shared Key = 12

Bob's Shared Key = 12

Key Exchange Successful!

=== Code Execution Successful ===

RESULT:

Thus, the Diffie–Hellman key exchange algorithm was successfully implemented in C. Alice and Bob generated their public keys and successfully established the same shared secret key, 12, without directly transmitting the secret key.